

 ХАКАТОН «ТОП ИТ» 2026 • КОМАНДА «КОД В САПОГЕ»

Executor

Balancer

Событийно-ориентированная система интеллектуального распределения входящих заявок высокой нагрузки между исполнителями

 4 000 заявок/час

 Двухфазные Lua-резервации

 CatBoost + Эвристический fallback

Вызовы и специфика ТЗ

Высокая нагрузка и лаг АИС

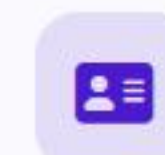
 **Поток до 4 000 заявок в час:** интенсивный входной трафик с периодическими пиковыми всплесками (bursts).

 **Задержка внешней АИС (2–10 сек):** медленная синхронная фиксация назначений во внешней системе.

 **Риск состояния гонки (Race Condition):** при параллельной обработке несколько воркеров могут назначить одну заявку или перегрузить одного исполнителя.

 **Ограничение слотов:** необходимость моментального резервирования емкости специалиста без блокировки всей очереди.

Многофакторность и баланс

 **Многомерные профили:** более 15 параметров у каждой заявки и свыше 20 настраиваемых атрибутов исполнителей.

 **Справедливое распределение (Fairness):** требование соблюдения баланса нагрузки с отклонением не более $\pm 1-2\%$.

 **Динамические правила:** изменение условий маршрутизации на лету без остановки сервисов и перекомпиляции ядра.

 **Трассируемость решений:** фиксация полного снимка отбора (Decision Trace) для прозрачного аудита назначений.

Масштаб задачи в цифрах

4 000

заявок в час
целевой входной поток по ТЗ

2–10 с



Задержка подтверждения АИС

Требует распределенной атомарной изоляции рабочего слота исполнителя на время HTTP-ответа.

35+



Параметров в оценке

Комплексный учет веса задачи, суммы, навыков, грейда специалиста, суточных квот и VIP-статуса.

±1–2%



Погрешность справедливости

Сбалансированное распределение совокупного веса открытых задач между всеми доступными слотами.

100%



Идемпотентность потока

Исключение дублей заявок через таблицу processed_events и уникальные idempotency-ключи.

Архитектурный ответ

Событийное ядро и разделение зон



Шина Apache Kafka: надежная буферизация пиков нагрузки и изоляция медленной АИС от вычислительного конвейера.



Независимые Consumer Groups: параллельное чтение событий Go-оркестратором и расчетным модулем Decision Engine.



Разделение данных: оперативный runtime-кэш в Redis 7 и долговременное персистентное хранение аудита в PostgreSQL 16.



Компенсация сбоев: автоматическая реконсиляция конфликтов 409 Conflict и очереди недоставленных сообщений (DLQ).

Двухфазное резервирование слотов



Фаза 1 (TryReserve): атомарный Redis Lua-скрипт проверяет лимиты и блокирует исполнителя со статусом PENDING.



Фаза 2 (Confirm / Cancel): после ответа внешней АИС бронь подтверждается либо моментально возвращается в пул.

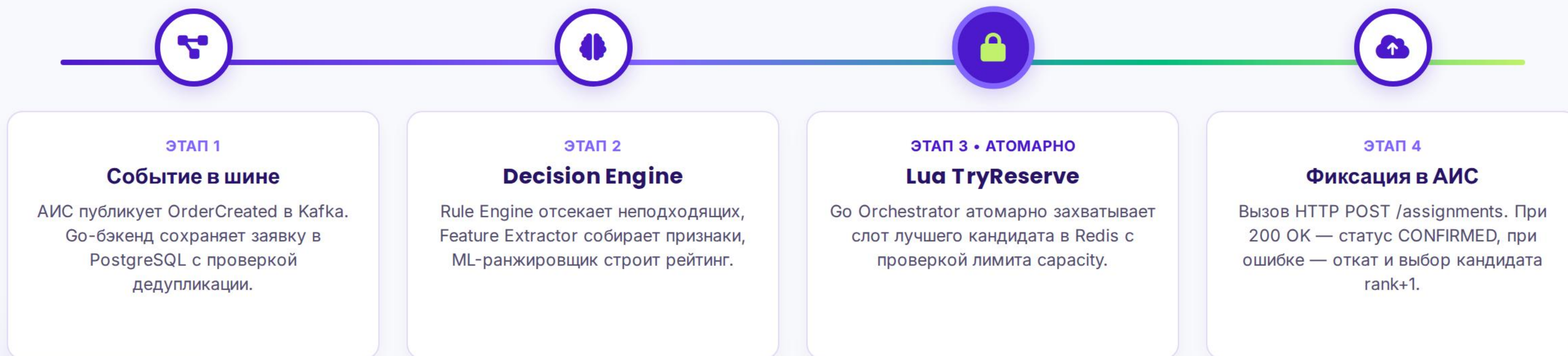


Контроль TTL брони: автоматическое освобождение зависших слотов воркером CancelExpired при падении узлов.



Многоуровневый скоринг: сочетание жестких AST-фильтров, ML-ранжировщика и балансировки по effective_load.

Сквозной конвейер заявки



Технологический стек

Слой системы	Технологии	Роль в проекте	Архитектурное преимущество
Backend Core	 Go 1.25 pgx, franz-go	Оркестрация назначения, Lua TryReserve, аудит решений	Минимальное потребление памяти, строгая модель конкурентности
Decision Engine	 Python 3.11 CatBoost, NumPy	Пайплайн правил, ML-ранжирование, расчет EffectiveLoad	Эластичность в обработке признаков, надежный fallback
Runtime State	 Redis 7 Lua Scripts, Hashes	Активные исполнители, мьютексы брони, счетчики нагрузки	Субмиллисекундная задержка, атомарность операций без гонок
Audit & Database	 PostgreSQL 16 JSONB	Проекция заявок, исполнителей, история назначений и трейсы	Надежность ACID, частичные уникальные индексы броней
Event Streaming	 Apache Kafka 3.9 KRaft	Асинхронный обмен событиями, изоляция отказов и DLQ	Высокая пропускная способность, отсутствие единой точки отказа

Модуль: Decision Engine

ШАГ 1



1. Rule Engine

Исключает нерелевантных кандидатов по системным и динамическим AST-правилам: активность, диапазоны сумм, суточные лимиты, типы клиентов (МСП/Крупный бизнес) и VIP-теги.

✓ Жесткая фильтрация пула

ШАГ 2



2. Feature & ML Ranker

Извлекает текстовые эмбединги задачи, сопоставляет навыки исполнителя (Skill Matching) и рассчитывает скор релевантности модели CatBoost по 19 признакам с fallback на эвристику.

✓ Качественный подбор исполнителя

ШАГ 3



3. Runtime Balancer

Считывает текущую нагрузку из Redis пакетом HGETALL и выстраивает финишный рейтинг кандидатов с приоритетом наименее загруженных и детерминированным тай-брейкером.

✓ Баланс нагрузки без перегрузок

Модуль: Go Orchestrator

Атомарные Lua-резервации

 **Скрипт TryReserve:** за одну неделимую операцию в памяти Redis проверяет условие `current_load + order.weight ≤ capacity` и ставит мьютекс.

 **Защита от двойного назначения:** временный ключ `order:reservation:{id}` блокирует параллельные попытки других воркеров.

 **Управление временем жизни (TTL):** бронь удерживается до 30 секунд; перед запросом в АИС выполняется refresh метки.

 **Автоматическая очистка:** фоновый воркер удаляет просроченные брони и возвращает слоты при аварии сетевого канала.

Надежность и устойчивость к сбоям

 **Идемпотентность:** каждое входящее Kafka-сообщение регистрируется в таблице `processed_events` с ручным коммитом offset.

 **Реконсиляция конфликтов 409:** при несовпадении состояния Go запрашивает факт из АИС и выравнивает статус в PostgreSQL.

 **Политика повторов (Retry):** повторные попытки вызова внешней системы через 500 мс и 1000 мс при временных сетевых ошибках.

 **Аудит Decision Trace:** сохранение причин выбора, значений признаков и времени принятия решения для разбора инцидентов.

АИС Симулятор и брокер



Генератор нагрузки

Режимы `stream_4k` (1 заявка в 0.9 сек — честные 4000 з/ч), `wave` и `spikes` для стресс-тестирования буферизации шины и поведения очередей.

► Эмуляция реального трафика



Жизненный цикл заявок

Реалистичная симуляция исходов: 55% асерт, 25% reject, 20% await. Автогенерация дочерних заявок с сохранением ссылки `parent_id`.

→ Освобождение рабочих слотов



Kafka KRaft и DLQ

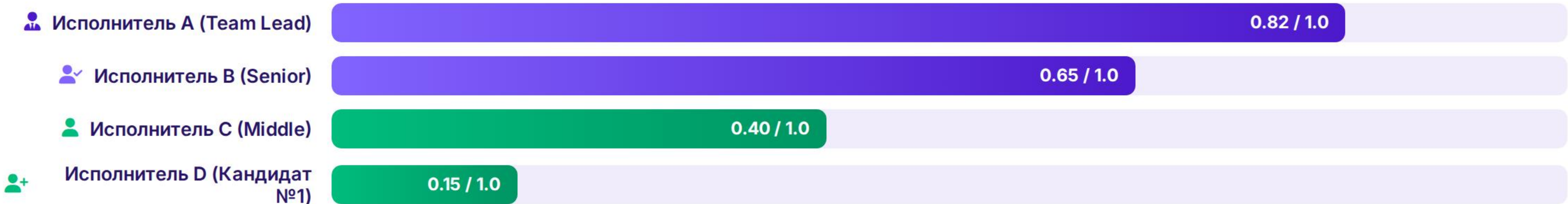
Изолированные топики для заявок, профилей исполнителей и решений. Выделенные Dead Letter Queues для изоляции поврежденных сообщений без остановки сервиса.

♥ Защита от сбоев доставки

Алгоритм балансировки

Расчет эффективной нагрузки (Effective Load):

$$\text{effective_load} = \frac{W_{\text{active}} + W_{\text{pending}}}{\text{capacity}}$$



6-уровневый тай-брейкер ранжирования: (1) min effective_load → (2) min открытых задач → (3) max ML score → (4) min выполненных за сутки → (5) наибольшее время с последнего назначения → (6) детерминированный ID.

Команда «Код в сапоге»



Backend Lead

Разработка Go Orchestrator, реализация атомарных Lua-скриптов в Redis, контракты Kafka и обработка отказов.

✓ Архитектура ядра



ML & Decision

Проектирование конвейера Decision Engine, AST-парсинг динамических правил, CatBoost ранжирование и метрики.

✓ Алгоритмы подбора



DevOps & System

Настройка кластера Kafka, схема PostgreSQL 16, симулятор АИС с генерацией 4k/час и интеграционные E2E тесты.

✓ Надежность стека



Product & Analytics

Бизнес-анализ требований ТЗ, конструктор бизнес-правил, прототип аналитического дашборда и экспорт метрик.

✓ Продуктовый контур

I Итоги и ответы на вопросы

Готовы к вопросам

Платформа Executor Balancer обеспечивает справедливое и отказоустойчивое распределение

✓ Поток 4 000 заявок/час

✓ Нулевой риск гонок в Redis

✓ Полный аудит Decision Trace

Команда «Код в сапоге» • Хакатон «ТОП ИТ» 2026